



+7 (495) 41-41-121

info@python-school.ru



Вы тут: [ГЛАВНАЯ](#) / [БЛОГ](#) / [NLP](#) / [ПРЕДОБРАБОТКА ТЕКСТА В NLP](#)

15
ИЮЛ

ПРЕДОБРАБОТКА ТЕКСТА В NLP

Автор [Роман Котюбеев](#)



В [предыдущий раз](#) мы говорили о векторизации текстовых данных в [NLP](#). Однако прежде чем преобразовать слова в числа, их следует обработать. Читайте в нашей статье о методах предобработки текста: токенизации, удалении стоп-слов, [стемминге](#) и лемматизации с Python-библиотеками [pymorphy2](#) и [NLTK](#).

РАЗБИВАЕМ ТЕКСТ НА ТОКЕНЫ

Токенизация – процесс разбиения текста на текстовые единицы, например, слова или предложения. В случае разбиений на предложения задача кажется тривиальной, нужно просто найти точку, вопросительный или восклицательный знак. Но в русском языке существует сокращения, в которых есть точка, например, *к.т.н.* — кандидат технических наук или *т.е.* — то есть. Поэтому такой путь может привести к ошибкам. К счастью, Python-библиотека **NLTK** позволяет избежать этой проблемы. Рассмотрим пример:

```
>>> from nltk.tokenize import sent_tokenize
>>> text = "Я - к.т.н, т.е. проучился долгое время. Имею образование."
>>> sent_tokenize(text, language="russian")
['Я - к.т.н, т.е. проучился долгое время.',
 'Имею образование.']
```

Как видим, функция `sent_tokenize` разбила исходное предложения на два, несмотря на присутствие слов *к.т.н.* и *т.е.*

Помимо разбиения на предложения в **NLTK** можно в качестве токенов использовать слова:

```
>>> from nltk.tokenize import sent_tokenize, word_tokenize
>>> text = "Я - к.т.н. Сажу на диван-кровать."
>>> word_tokenize(text, language="russian")
['Я', '-', 'к.т.н.', 'Сажу', 'на', 'диван-кровать', '.']
```

Здесь *к.т.н.* и *диван-кровать* были определены как отдельные слова.

ИСКЛЮЧАЕМ СТОП-СЛОВА ИЗ ИСХОДНОГО ТЕКСТА

Иногда одних слов в тексте больше, чем других, к тому же они встречаются почти в каждом предложении и не несут большой информативной нагрузки.

Такие слова являются шумом для последующего глубокого обучения (**Deep Learning**) и называются **стоп-словами**. Библиотека **NLTK** также имеет список стоп-слов, который предварительно необходимо скачать. Это можно сделать следующим образом:

```
>>> import nltk
>>> nltk.download('stopwords')
```

После этого доступен список стоп-слов для русского языка:

```
>>> from nltk.corpus import stopwords
>>> stopwords.words("russian")
```

Всего их насчитывается в этом списке 151. Вот некоторые из них:

```
и, в, во, не, что, он, на, я, с, со, как, а,
то, все, чтоб, без, будто, впрочем, хорошо,
свою, этой, перед, иногда, лучше, чуть, том,
нельзя, такой, им, более, всегда, конечно,
всю, между
```

Поскольку это список, то к нему можно добавить дополнительные слова или, наоборот, удалить из него те, которые будут информативными для вашего случая. Для последующего исключения слов из токенизированного текста можно написать следующее:

```
for token in tokens:
    if token not in stop_words:
        filtered_tokens.append(token)
```

СТЕММИНГ: УДАЛЯЕМ ОКОНЧАНИЯ

Русский язык обладает богатой морфологической структурой. Слово *хороший* и *хорошая* имеют тот же смысл, но разную форму, например, *хорошая мебель* и *хороший стул*. Поэтому для машинного обучения (*Machine Learning*) лучше привести их к одной форме для уменьшения размерности. Одним из таких методов является *стеemming* (stemming). В частности, он опускает окончания слова. В Python-библиотеке *NLTK* для этого есть `Snowball Stemmer`, который поддерживает русский язык:

```
>>> from nltk.stem import SnowballStemmer
...
>>> snowball =
SnowballStemmer(language="russian")
>>> snowball.stem("Хороший")
хорош
>>> snowball.stem("Хорошая")
хорош
```

Проблемы могут возникнуть со словами, которые значительно изменяются в других формах:

```
>>> snowball.stem("Хочу")
хоч
>>> snowball.stem("Хотеть")
хотет
```

Хотеть и *хочу* — грамматические формы одного и то же слова, но *стеemming* обрубаёт окончания согласно своему алгоритму. Поэтому возможно следует применить другой метод — лемматизацию.

ПРИВЕДЕНИЕ К НАЧАЛЬНОЙ ФОРМЕ С ЛЕММАТИЗАЦИЕЙ

Над словом можно провести морфологический анализ и выявить его начальную форму. Например, *хочу*, *хотят*, *хотели* имеют начальную форму *хотеть*. Тогда можем воспользоваться *py morphology2* — инструмент для морфологического анализа русского и украинского языков.

Рассмотрим пример для слова “хочу”:

```
>>> import pymorphy2
>>> morph = pymorphy2.MorphAnalyzer()
>>> morph.parse("хочу")
[Parse(word='хочу',
tag=OpencorporaTag('VERB,impf,tran
sing,1per,pres,indc'), normal_form='хотеть',
score=1.0, methods_stack=
((DictionaryAnalyzer, 'хочу', 2999, 1),))]
```

Метод `parse` возвращает список объектов `Parse`, которые обозначают виды грамматических форм анализируемого слова. Такой объект обладает следующими атрибутами:

`tag` обозначает набор грамем. В данном случае слово `хочу` — это глагол (VERB) несовершенного вида (impf), переходный (tran), единственного числа (sing), 1 лица (1per), настоящего времени (pres), изъявительного наклонения (indc);

`normal_form` — нормальная форма слова;

`score` — оценка вероятности того, что данный разбор правильный;

`methods_stack` — тип словаря распарсенного слова с его индексом.

Нас больше всего интересует нормальная форма слова. По умолчанию объекты `Parse` сортированы в порядке убывания значения `score`. Поэтому из списка лучше всего брать 1-й элемент:

```
>>> morph.parse("хотеть")[0].normal_form
хотеть
>>> morph.parse("хочу")[0].normal_form
хотеть
>>> morph.parse("хотят")[0].normal_form
хотеть
```

Таким образом, мы получили одно слово из разных его форм.

Как обрабатывать текстовые данные и применять их в реальных проектах Data Science для моделей **Machine Learning** с помощью Python, вы узнаете на наших курсах в лицензированном учебном центре обучения и повышения квалификации ИТ-специалистов в Москве.

[Смотреть расписание](#)[Записаться на курс](#)

Tag: nltk, текстовые данные



ПОХОЖИЕ ПУБЛИКАЦИИ

19 апреля, 2022

ЧТО НАМ ИЗВЕСТНО О ВЫХОДЯЩЕМ GPT-4?

15 февраля, 2022

ГРАФ ЗНАНИЙ

ДОБАВИТЬ КОММЕНТАРИЙ

Для отправки комментария вам необходимо [авторизоваться](#).

[Главная](#)[test-code](#)[Блог](#)[Контакты](#)

МЕТКИ

best practice (110)

cnn (1)

data augmentation (4)

datetime (1)

Dict comprehension (1)

docker (1)

GPU (3)

IDE (2)

Iterable (5)

Jupyter (57)

List comprehension (5)

matplotlib (5)

MLOps (4)

neural network (14)

nlTK (2)

numpy (9)

Nvidia Jetson Nano (4)

os (2)

pandas (13)

pathlib (2)

Показать все метки

КАЛЕНДАРЬ ЗАПИСЕЙ

Октябрь 2025

Пн	Вт	Ср	Чт	Пт	Сб	Вс
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

« Фев

СВЕЖИЕ КОММЕНТАРИИ

[Как найти скользящую среднюю в PySpark к записи](#)

5 графиков для Data Science, которые можно построить в Pandas 3 способами

Функции PySpark SQL для парсинга JSON формата к записи

Как сериализовать и десериализовать JSON в Python

Роман Котюбеев к записи

5 ключевых достоинств и 3 главных недостатка Python

Роман Котюбеев к записи

Решаем задачу численного прогнозирования с помощью линейной регрессии на Python

Sergey к записи

Решаем задачу численного прогнозирования с помощью линейной регрессии на Python

АРХИВ

Выберите месяц



© 2021-2025 ООО «УЦ «Коммерсант». Все права защищены.

Мы используем файлы cookie Это поможет Вам эффективно использовать Сайт, а нам поддерживать корректное функционирование его сервисов. Подтвердите свое согласие на обработку cookie, приняв условия, согласно [политике использования файлов cookie](#) или покиньте сайт

ПРИНИМАЮ